RB2301 CA1 Report

No.	Name	Student ID
1.	Maxwell Timothy Ebora Guico	A0323869N
2.	Suda Jesmitha Reddy	A0321806L
3.	Nguyen Linh Nhien	A0325663B

PROBLEM: Obstacle Avoidance

Given a nano-car robot with 2D movement that repeatedly returns 721 data points (360-degree) from its LiDAR sensor. The robot spawns at the bottom edge of a map randomly filled with cylindrical obstacles and must be programmed so that it is able to traverse to the top edge while reactively avoiding the cans.

ALGORITHMS

Our group implemented 3 different algorithms to solve the obstacle course. This report will discuss Nhien's algorithm

Map Boundary detection / centering algorithm

```
self.last_scan = np.clip(self.last_scan, 0, 20) # Clip the scanned values to avoid inf and 0

last_scan_copy = self.last_scan.copy()
    #Split into 2 parts to prevent over detecting edge (>180)
    #left first
window_size = 90
window_start = 0
while window_start <= len(self.last_scan)/2 - window_size:
    window_end = window_start + window_size
    if np.all(self.last_scan[window_start:window_end] == 20):
        while window_end < len(self.last_scan)/2 and self.last_scan[window_end] == 20:
            window_end += 1
        angle_degrees = window_end - window_start
        self.last_scan[window_start:window_end] = boundary_distance(
            angle_degrees)
        window_start = window_end
        #print("Zeroed left")
        continue
    window_start += 1

#Right
while window_start <= len(self.last_scan) - window_size:
    window_end = window_start + window_size
    if np.all(self.last_scan[window_start:window_end] == 20):
        while window_end < len(self.last_scan) and self.last_scan[window_end] == 20:
            window_end += 1
        angle_degrees = window_end - window_start
        self.last_scan[window_start:window_end] = boundary_distance(
            angle_degrees)
        window_start = window_end
        #print("Zeroed right")
        continue
    window_start += 1
```

Fig 1. Code for Gap Scoring

An expandable 90-degree moving window moves across each half of the scans array. Once all the elements in the scan array are inf (clipped to 20m), its start expanding forward until it reaches a scan <20m. Then, the robot will estimate how far is it from the boundary and assign a new scan value X to the detected gap, calculated using a decaying exponential:

```python
def boundary_distance(angle_degrees):
    """Map boundary angular width to distance, quartering every 30 degrees."""
    return 20.0 * 4.0 ** (-(angle_degrees - 90.0) / 30.0)
```

Fig 2. Decaying exponential function for estimating boundary distance

This means the closer it gets to the boundary (larger gap on either left or right from 90); it will assign a new value starting with 20m for 90 degrees, to ~0.35 for 180 degrees. This acts as the centering algorithm

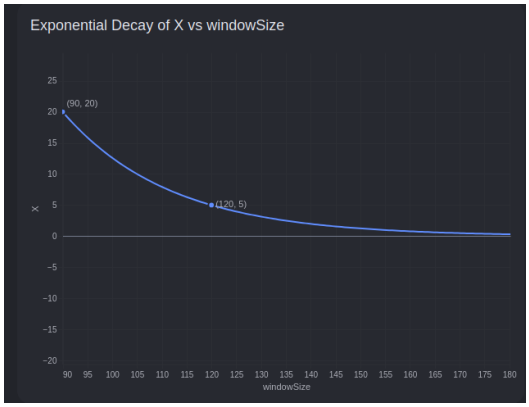


Fig 3. X vs Window size relationship

Gap detection and scoring

```python
self.last_scan = np.clip(np.concatenate((self.last_scan, self.last_scan)), 0, 20)
window_size = 20
window_start = 0
dir  = 0
best_gap_score = 0
best_gap_center = 0
while window_start <= len(self.last_scan) - window_size:
    window_end = window_start + window_size
    if np.all(self.last_scan[window_start:window_end] >= 0.5):
        while (window_end < len(self.last_scan)) and (self.last_scan[window_end] >= 0.5):
            window_end += 1
        gap_width = window_end - window_start
        gap_distance_values = self.last_scan[window_start:window_end]
        mean_distance = float(np.mean(gap_distance_values)) if gap_distance_values.size > 0 else 0.0
        self.last_scan[window_start:window_end] = mean_distance
        if gap_  (variable) mean_distance: float
            if mean_distance >= 2:
                gap_score = (gap_width + mean_distance) / 2
                if gap_score > best_gap_score:
                    best_gap_score = gap_score
                    best_gap_center = window_start + (window_end - window_start) / 2
        window_start = window_end
        window_size = 20
        continue
    else:
        self.last_scan[window_start:window_end] = 0
        window_start = window_end

if best_gap_score > 0:
    dir = best_gap_center
else:
    dir = 0
```

Fig 4. Gap Scoring mechanism

After getting the new scan array, the robot duplicates and concatenates it to make a circular scan array.

A moving window with size 10 moves through the circular array and starts expanding whenever all its elements >=0.5m, until it reaches a scan < 0.5m. Then it scores the gap by averaging its width and mean gap

distance. If the score is highest, save the score for later comparison, and save the direction of its middle scan direction for movement

Finally, the robot moves to the highest scoring gap it finds.

Emergency Reaction

```
if np.any(last_scan_copy[90:181]<0.25):
    self.tempo_blind_left += 10
    self.tempo_blind_right = 0
    print("LEFTDOWN")
elif np.any(last_scan_copy[181:271]<0.25):
    self.tempo_blind_right += 10
    self.tempo_blind_left = 0
    print("RIGHTDOWN")

if (np.any(last_scan_copy[-180:]<0.25) and self.tempo_blind_left == 0):
    print("Emergency left")
    self.tempo_blind_right += 10
    delta_x = delta_x
    delta_y = (delta_y if delta_y > 0 else 0.2)
elif (np.any(last_scan_copy[:180]<0.25) and self.tempo_blind_right == 0):
    print("Emergency right")
    self.tempo_blind_left += 10
    delta_x = delta_x
    delta_y = (delta_y if delta_y < 0 else -0.2)

#GO BACK
if np.any(last_scan_copy[-90:] < 0.15) or np.any(last_scan_copy[:90] < 0.15):
    print("Emergency BACK")
    delta_x = (delta_x if delta_x < 0 else 0)
    delta_y = delta_y
```

Fig 5. Emergency case handler

After getting the movement direction from the “best” gap, the robot needs to react to obstacles too close to it using primary movements.

If obstacle too close on the right => only allow left movement; If obstacle too close on the right => only allow left movement; If obstacle too close in front => Cancel front movement

Directional consistency

```
#Mask the back
self.last_scan[100:260:5] = 0.1

#Blind left/ right side
if self.tempo_blind_left > 0:
    self.tempo_blind_left -= 1
    self.last_scan[0:180:5] = 0.1
    self.tempo_blind_right = 0
    print("Blind left")

if self.tempo_blind_right > 0:
    self.tempo_blind_right -= 1
    self.last_scan[-180::5]  = 0.1
    self.tempo_blind_right = 0
    print("Blind right")
```

Fig 5. Emergency case handler

This is for the L-shape map edge case, when the robot detects a right obstacle too close in its back-right/back-left 90 degrees, indicating a potential wall on the left, it sets a timer to partially blind itself on the corresponding left/right side to not detect any gap in that region later (using a 5-step mask, the window size 10 will not be able to detect this as gap). This helps the robot to consistently travel in the other direction of the wall.

EMPIRICAL RESULTS

The figure below are the simulated run times in seconds, path length taken by the robot in meters, the number of cans present and whether the run was successful. Height, width, edge case spacing and size_div were kept constant with the seed value being the only variable changed. The following are the parameters in the obstacle_generator.py

Seed	Time (simulated s)	Path length (m)	Cans	Success
1	18.850	7.285	46	Yes
2	20.000	7.639	43	Yes
3	20.400	7.822	40	Yes
4	18.300	7.134	44	Yes
5	21.750	7.919	42	Yes
Total	99.300	37.799	—	**5/5**
Average	**19.86**	**7.5598**	—	—

	Number of runs	Success Rate (%)	Average Time Taken (s)	Average Path Length (m)
algo = nhien	5	100	19.86	7.5598
algo = max	5	100	26.00	7.249

Comparing the two algorithms, algo = nhien achieved a lower average completion time of 19.86 s, compared to 26.00 s for algo = max. However, algo = max produced a slightly shorter average path length of 7.249 m, compared to 7.5598 m for algo = nhien. Both algorithms achieved a 100% success rate across five runs. The difference in completion time may be partly attributed to the different movement speeds used by the two algorithms, while the difference in path length reflects variations in their obstacle-avoidance and path-selection strategies.

REFLECTION

This projects enable us to think consistently while also keeping track of multiple aspects of a robotics project. We started with a simple 4-direction algorithm where the robot just reacts according to which direction is the furthest obstacle away. As we move on, our ambition grows; we start trying to make it move more robustly, in all directions. We tried 2 different algorithms: “Gap Detection using Vectors” (Max) and “Gap Scoring and reactive direction bias” (Nhien). Next, we start trying to code the launch file to make the code run the simulation, the algorithm, and the grader all run in one launch line instead of requiring multiple terminals. We’ve come a long way from if else’s to actually using math and transformation.

For the algorithm discussed in this report, the first problem lies between enabling centering and L-shape algorithm, as the L-shape requires us to travel slightly pass the map boundary (In this case determined by the robot), while the centering requires the robot to be more sensitive. The next problem comes from setting the right distance value for what is considered obstacles; this is mostly due to the scan array being polar, and the LiDAR placement not being exactly at the robot center.

For real life testing, it is much more complicated. We ran our code and the robot ramped straight into the cylinders. That's when we found out about the LiDAR offset and its 5cm limitation. And we tried to overcome it by resetting scanning parameters, lowering speed, which increased our likelihood of success by a significant amount.

After the CA1, we would like to explore other existing reactive solutions, different movement mechanisms, and utilize the template Gazebo simulation given in this lab to test.

Nguyen Linh Nhien's AI Declaration

I have used GPT-5.6-Sol to help me create auto-grader.py, modify launch file codes, and implement node parameters for combining the 2 algorithms, and debugging errors related to Gazebo Simulation when large changes are made.